

Software Build Automation Tools — Module 1 Project

Git as a “Time Machine” • Project: Monthly Budget Planner

Author: Mustafa Cankatar | Repository: <https://github.com/mustafacankatarr/Monthly-Budget-Planner>

1. Purpose of the Script

Monthly Budget Planner is a small command-line Python application that reads a list of monthly expenses from an external file (expenses.txt) and writes a formatted spending report to an output file (report.txt). On each run it calculates the total spending, a per-category breakdown showing each category’s share of the total, the average daily spending, a recommended emergency fund (a configurable percentage of total spending), and the single biggest expense. The input parser is deliberately robust: blank lines, comment lines (starting with #) and malformed entries are skipped with a warning, so a single typo cannot crash the whole report. The script demonstrates the core build-automation idea of a reproducible process that turns a tracked input file into a generated, non-tracked output file.

2. Git Evolution (11 commits, a branch merge and a resolved conflict)

The project was built incrementally through 11 commits (the minimum required is 5), each with a clear “what and why” message: read the input and compute the total; add a per-category breakdown; calculate the emergency fund and write report.txt; add a .gitignore; add a README; make the parser robust; sort the breakdown and show percentage shares; and add average-daily-spending and biggest-expense statistics.

A feature branch, feature-emergency-fund, was then created to raise the emergency-fund buffer from 10% to 15%, while the main branch independently raised the same value to 20%. Merging the branch into main therefore produced a deliberate merge conflict on the BUFFER_RATE line. The conflict was resolved manually by keeping the 20% value, which created the final merge commit, and the release was tagged v1.0. The generated file report.txt is excluded by .gitignore, so it never appears as a tracked change.

3. Evidence

```
git log --graph --oneline --all --decorate
* 0cf7206 (HEAD -> main, tag: v1.0) Merge feature-emergency-fund and resolve conflict: keep emergency fund buffer at 20%
|\
| * 41d1d87 (feature-emergency-fund) Increase emergency fund buffer from 10% to 15%
* | c5bc3d4 Increase emergency fund buffer from 10% to 20%
|/
* 78f1800 Add average daily spending and biggest-expense highlight to the report
* 053f65e Sort the breakdown by amount and show the percentage share of total
* e26d7c9 Make the parser robust: skip comments, blanks and bad entries
* 29b6d66 Add README with project description and usage instructions
* 4505c2e Add .gitignore to exclude the generated report and Python cache
* 01f8a76 Calculate emergency fund (10%) and write the summary to report.txt
* dfbcab4 Add per-category spending breakdown to the summary
* 258d532 Initial commit: read expenses from file and calculate total spending
```

Figure 1 — Full commit history and branch/merge structure (git log --graph --oneline --all --decorate).

```
●●● Merge conflict markers & final merge commit

$ git merge feature-emergency-fund
Auto-merging budget.py
CONFLICT (content): Merge conflict in budget.py
Automatic merge failed; fix conflicts and then commit the result.

$ cat budget.py (lines 14-20 - the conflict)
# Share of monthly spending to set aside as an emergency fund.
<<<<<<< HEAD
BUFFER_RATE = 0.20
=====
BUFFER_RATE = 0.15
>>>>>> feature-emergency-fund

$ git log --oneline -3 (after resolving the conflict)
0cf7206 Merge feature-emergency-fund and resolve conflict: keep emergency fund buffer at 20%
c5bc3d4 Increase emergency fund buffer from 10% to 20%
41d1d87 Increase emergency fund buffer from 10% to 15%
```

Figure 2 — The conflict markers (<<<<<<<, =====, >>>>>>>) and the final merge commit after resolution.

```
●●● git status - report.txt is ignored

$ python budget.py (regenerates report.txt)
Report written to report.txt

$ git status
On branch main
nothing to commit, working tree clean

$ git status --ignored
On branch main
Ignored files:
  (use "git add -f <file>..." to include in what will be committed)
    report.txt

nothing to commit, working tree clean
```

Figure 3 — git status confirming the output file report.txt is ignored by Git.